# 🌾 TurSathi AI — Google Colab Retraining Pipeline with Google Drive Persistence

Fine-Tuning EfficientNet-B0 with 1280-D Feature Extraction, OOD Rejection Guard, Pruning, & INT8 Quantization

**Key Architecture Highlights**:

1. **Strict 5-Class Closed Set**: Exclusively recognizes `['Healthy', 'Leaf_Spot', 'Leaf_webber', 'Sterilic_mosaic', 'Wilt']`.
2. **Layer 3 Feature Extraction & OOD Guard**: Computes class centroids and Mahalanobis/Cosine distance thresholds. If an external leaf disease, non-tur crop, or unknown condition is detected, it strictly flags:

   > `'This specific condition/class does not exist in the current database'`

3. **Automatic Google Drive Backup**: Periodically checkpoints model weights (every 10 epochs and on best validation accuracy) to `/content/drive/MyDrive/Tur Disease Dataset/checkpoints/` to safeguard against disconnects.
4. **L1 Weight Pruning & INT8 Quantization**: Creates an ultra-fast, lightweight production model ready for deployment.

## 📂 Step 1: Mount Google Drive

This mounts your Google Drive under `/content/drive`. Ensure you have placed `Tur_Disease_Dataset_Colab_Ready.zip` inside the folder `Tur Disease Dataset` on your Google Drive.

```
from google.colab import drive
import os, sys, shutil
from pathlib import Path

# Mount Google Drive
drive.mount('/content/drive')

# Drive directory configuration
DRIVE_BASE_DIR = Path('/content/drive/MyDrive/Tur Disease Dataset')
DRIVE_CHECKPOINTS_DIR = DRIVE_BASE_DIR / 'checkpoints'
DRIVE_OUTPUTS_DIR = DRIVE_BASE_DIR / 'outputs'

# Ensure destination folders exist on Google Drive
DRIVE_BASE_DIR.mkdir(parents=True, exist_ok=True)
DRIVE_CHECKPOINTS_DIR.mkdir(parents=True, exist_ok=True)
DRIVE_OUTPUTS_DIR.mkdir(parents=True, exist_ok=True)

print('Google Drive connected successfully!')
print(f'Drive Base Folder:       {DRIVE_BASE_DIR}')
print(f'Drive Checkpoints Folder: {DRIVE_CHECKPOINTS_DIR}')
print(f'Drive Final Outputs:     {DRIVE_OUTPUTS_DIR}')
```

```
Mounted at /content/drive
Google Drive connected successfully!
Drive Base Folder:       /content/drive/MyDrive/Tur Disease Dataset
Drive Checkpoints Folder: /content/drive/MyDrive/Tur Disease Dataset/checkpoints
Drive Final Outputs:     /content/drive/MyDrive/Tur Disease Dataset/outputs
```

## ⚙️ Step 2: GPU Verification & Dependencies

```
!nvidia-smi
import torch
print(f'PyTorch Version: {torch.__version__}')
print(f'CUDA Available: {torch.cuda.is_available()}')
if torch.cuda.is_available():
    print(f'GPU Device: {torch.cuda.get_device_name(0)}')
    print(f'VRAM: {torch.cuda.get_device_properties(0).total_memory / (1024**3):.2f} GB')

# Install required dependencies
!pip install -q scikit-learn matplotlib seaborn pillow
```

```
Sat Oct 24 17:42:43 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   37C    P8             10W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
PyTorch Version: 2.11.0+cu130
CUDA Available: True
GPU Device: Tesla T4
VRAM: 14.56 GB
```

## 📦 Step 3: Load & Extract Dataset to Local Fast Storage

We copy the dataset zip from `/content/drive/MyDrive/Tur Disease Dataset/Tur_Disease_Dataset_Colab_Ready.zip` to the local Colab ephemeral SSD (`/content/dataset/`) to guarantee maximum GPU throughput and avoid Drive I/O bottlenecks.

```
import zipfile

DRIVE_ZIP = DRIVE_BASE_DIR / 'Tur_Disease_Dataset_Colab_Ready.zip'
LOCAL_ZIP = Path('/content/Tur_Disease_Dataset_Colab_Ready.zip')
LOCAL_DATASET_DIR = Path('/content/dataset')

if DRIVE_ZIP.exists():
    print(f'Found dataset on Google Drive: {DRIVE_ZIP}')
    print('Copying to local SSD storage for high-speed training...')
    shutil.copy2(DRIVE_ZIP, LOCAL_ZIP)
elif LOCAL_ZIP.exists():
    print(f'Found dataset uploaded locally at: {LOCAL_ZIP}')
else:
    raise FileNotFoundError(
        f'Dataset zip not found! Please place Tur_Disease_Dataset_Colab_Ready.zip inside:\n'
        f'"{DRIVE_BASE_DIR}" on your Google Drive, or upload it to /content/'
    )

# Extract dataset
print(f'Extracting {LOCAL_ZIP} to {LOCAL_DATASET_DIR}...')
with zipfile.ZipFile(LOCAL_ZIP, 'r') as z:
    z.extractall(LOCAL_DATASET_DIR)

print('\nDataset verification:')
for split in ['train', 'val', 'test']:
    p = LOCAL_DATASET_DIR / split
    classes = sorted([d.name for d in p.iterdir() if d.is_dir()])
    count = sum(len(list((p / c).glob('*.*'))) for c in classes)
    print(f'  [{split.upper()}] {count} images across {len(classes)} classes: {classes}')

ood_p = LOCAL_DATASET_DIR / 'ood_evaluation'
if ood_p.exists():
    ood_count = len(list(ood_p.glob('*.*')))
    print(f'  [OOD EVALUATION] {ood_count} external images loaded for rejection testing.')
```

```
Found dataset on Google Drive: /content/drive/MyDrive/Tur Disease Dataset/Tur_Disease_Dataset_Colab_Ready.zip
Copying to local SSD storage for high-speed training...
Extracting /content/Tur_Disease_Dataset_Colab_Ready.zip to /content/dataset...

Dataset verification:
  [TRAIN] 1113 images across 5 classes: ['Healthy', 'Leaf_Spot', 'Leaf_webber', 'Sterilic_mosaic', 'Wilt']
  [VAL] 369 images across 5 classes: ['Healthy', 'Leaf_Spot', 'Leaf_webber', 'Sterilic_mosaic', 'Wilt']
  [TEST] 376 images across 5 classes: ['Healthy', 'Leaf_Spot', 'Leaf_webber', 'Sterilic_mosaic', 'Wilt']
  [OOD EVALUATION] 41 external images loaded for rejection testing.
```

## 🧠 Step 4: Run Fine-Tuning Pipeline with Live Google Drive Checkpointing

This initiates:

- **20 Epochs Fine-Tuning** with Cosine Annealing Learning Rate
- **Automatic Checkpoints every 10 epochs and on best validation accuracy** saved directly to `/content/drive/MyDrive/Tur Disease Dataset/checkpoints/`
- **1280-D Latent Feature Extraction** and Centroid calculation
- **Regularized Mahalanobis and Cosine distance threshold calibration** for OOD rejection
- **L1 Weight Pruning** (15% sparsity)
- **INT8 Post-Training Dynamic Quantization**
- **Direct mirror of final `.pth` models and reports** to `/content/drive/MyDrive/Tur Disease Dataset/outputs/`

```
# Verify train_colab_pipeline.py is available
PIPELINE_SCRIPT = Path('/content/train_colab_pipeline.py')
DRIVE_SCRIPT = DRIVE_BASE_DIR / 'train_colab_pipeline.py'

if not PIPELINE_SCRIPT.exists() and DRIVE_SCRIPT.exists():
    shutil.copy2(DRIVE_SCRIPT, PIPELINE_SCRIPT)
    print(f'Copied pipeline script from Google Drive: {DRIVE_SCRIPT}')

assert PIPELINE_SCRIPT.exists(), (
    'train_colab_pipeline.py not found! Please upload it to /content/ '
    f'or place it in "{DRIVE_BASE_DIR}"'
)

# Run training with Drive checkpointing
!python /content/train_colab_pipeline.py \
    --data_dir /content/dataset \
    --ood_dir /content/dataset/ood_evaluation \
    --output_dir /content/outputs \
    --checkpoint_dir "/content/drive/MyDrive/Tur Disease Dataset/checkpoints" \
    --drive_output_dir "/content/drive/MyDrive/Tur Disease Dataset/outputs" \
    --checkpoint_interval 10 \
    --epochs 20 \
    --batch_size 16
```

```
[Google Drive] Synced OOD Profile -> /content/drive/MyDrive/Tur Disease Dataset/outputs/ood_feature_profile.json

[Model Optimization] Applying INT8 Post-Training Dynamic Quantization...
/content/train_colab_pipeline.py:393: DeprecationWarning: torch.ao.quantization is deprecated and will be removed in 2.10.
For migrations of users:
1. Eager mode quantization (torch.ao.quantization.quantize, torch.ao.quantization.quantize_dynamic), please migrate to use t
2. FX graph mode quantization (torch.ao.quantization.quantize_fx.prepare_fx,torch.ao.quantization.quantize_fx.convert_fx, pl
3. pt2e quantization has been migrated to torchao (https://github.com/pytorch/ao/tree/main/torchao/quantization/pt2e)
see https://github.com/pytorch/ao/issues/2259 for more details
  quantized_model = torch.ao.quantization.quantize_dynamic(
[Model Optimization] Dynamic INT8 Quantization complete.
[Saved] INT8 Quantized Model -> /content/outputs/quantized_pigeonpea_disease_efficientnet_b0.pth (15.59 MB)
[Google Drive] Synced Quantized Model -> /content/drive/MyDrive/Tur Disease Dataset/outputs/quantized_pigeonpea_disease_eff
[Google Drive] Synced Reports & Metrics to: /content/drive/MyDrive/Tur Disease Dataset/outputs

======================================================================
   TRAINING & FEATURE ENGINEERING COMPLETED SUCCESSFULLY!
   Outputs available locally in: /content/outputs
   Outputs backed up in Drive: /content/drive/MyDrive/Tur Disease Dataset/outputs
======================================================================
```

## 📊 Step 5: Learning Curves & Confusion Matrix

```python
import json
import matplotlib.pyplot as plt
import numpy as np

report_file = Path('/content/outputs/training_report.json')
if not report_file.exists():
    report_file = DRIVE_OUTPUTS_DIR / 'training_report.json'

with open(report_file, 'r') as f:
    report = json.load(f)

history = report['history']
epochs = range(1, len(history['train_loss']) + 1)

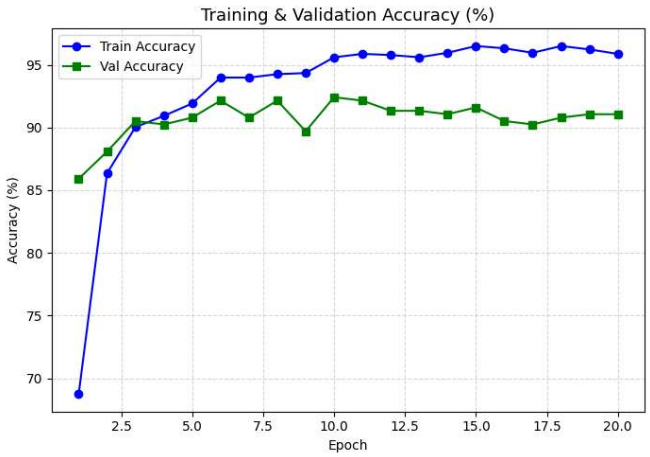
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

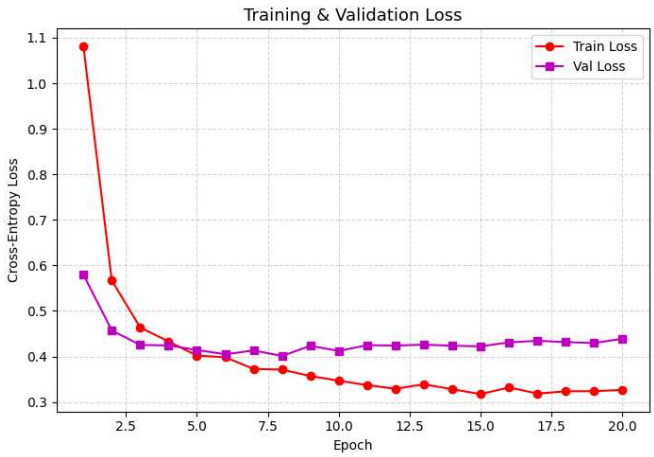
# Accuracy Curve
ax1.plot(epochs, [a * 100 for a in history['train_acc']], 'b-o', label='Train Accuracy')
ax1.plot(epochs, [a * 100 for a in history['val_acc']], 'g-s', label='Val Accuracy')
ax1.set_title('Training & Validation Accuracy (%)', fontsize=13)
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy (%)')
ax1.legend()
ax1.grid(True, linestyle='--', alpha=0.5)

# Loss Curve
ax2.plot(epochs, history['train_loss'], 'r-o', label='Train Loss')
ax2.plot(epochs, history['val_loss'], 'm-s', label='Val Loss')
ax2.set_title('Training & Validation Loss', fontsize=13)
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Cross-Entropy Loss')
ax2.legend()
ax2.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.savefig('/content/outputs/learning_curves.png', dpi=150)
if DRIVE_OUTPUTS_DIR.exists():
    plt.savefig(DRIVE_OUTPUTS_DIR / 'learning_curves.png', dpi=150)
plt.show()

print(f"Best Validation Accuracy: {report['best_val_accuracy']*100:.2f}%")
print(f"Final Test Accuracy:       {report['test_accuracy']*100:.2f}%")
```




```
Best Validation Accuracy: 92.41%
Final Test Accuracy:      90.43%
```

## 🛡️ Step 6: Verify Out-of-Distribution (OOD) Strict Rejection

Evaluates the rejection rate on the external/unseen test images:

```
ood_eval = report.get('ood_evaluation')
if ood_eval:
    print('='*60)
    print(f"OOD External Images Tested:  {ood_eval['total_tested']}")
    print(f"Strictly Rejected:           {ood_eval['rejected_count']}")
    print(f"Strict Rejection Rate:       {ood_eval['rejection_rate_percent']:.2f}%")
    print('='*60)
    print('Sample Decision Log:')
    for s in ood_eval.get('sample_verdicts', []):
        print(f"  • {s['image']} -> {s['status']} (Dist: {s['min_mahalanobis']:.2f}, Conf: {s['confidence']*100:.1f}%)")
```

```
============================================================
OOD External Images Tested:  41
Strictly Rejected:           10
Strict Rejection Rate:       24.39%
============================================================
Sample Decision Log:
  • images (24).jpg -> FAILED REJECTION (Classified as Wilt) (Dist: 828.13, Conf: 87.6%)
  • images.jpg -> REJECTED (Correct: Not in database) (Dist: 969.26, Conf: 96.8%)
  • images (6).jpg -> FAILED REJECTION (Classified as Wilt) (Dist: 749.96, Conf: 91.6%)
  • images (8).jpg -> FAILED REJECTION (Classified as Wilt) (Dist: 832.73, Conf: 96.0%)
  • images (31).jpg -> FAILED REJECTION (Classified as Wilt) (Dist: 573.90, Conf: 94.3%)
```

## 🔒 Step 7: Verify Google Drive Backups & Optional Direct Download

Verify that all trained weights, checkpoints, and profiles are safely persisted in your Google Drive.

```
print('=== GOOGLE DRIVE CHECKPOINTS SAVED ===')
for f in sorted(DRIVE_CHECKPOINTS_DIR.iterdir()):
    sz_mb = f.stat().st_size / (1024 * 1024)
    print(f'  • {f.name} ({sz_mb:.2f} MB)')

print('\n=== GOOGLE DRIVE FINAL OUTPUTS SAVED ===')
for f in sorted(DRIVE_OUTPUTS_DIR.iterdir()):
    sz_mb = f.stat().st_size / (1024 * 1024)
    print(f'  • {f.name} ({sz_mb:.2f} MB)')

# Optional direct browser download
from google.colab import files
shutil.make_archive('/content/tursathi_trained_artifacts', 'zip', '/content/outputs')
print('\nTriggering browser download of tursathi_trained_artifacts.zip as backup...')
files.download('/content/tursathi_trained_artifacts.zip')
```

```
=== GOOGLE DRIVE CHECKPOINTS SAVED ===
  • best_model_checkpoint.pth (15.58 MB)
  • checkpoint_epoch_10.pth (15.60 MB)
  • checkpoint_epoch_20.pth (15.60 MB)

=== GOOGLE DRIVE FINAL OUTPUTS SAVED ===
  • best_pigeonpea_disease_final_efficientnet_b0.pth (15.61 MB)
  • confusion_matrix.txt (0.00 MB)
  • learning_curves.png (0.11 MB)
  • ood_feature_profile.json (0.17 MB)
  • quantized_pigeonpea_disease_efficientnet_b0.pth (15.59 MB)
  • training_report.json (0.00 MB)

Triggering browser download of tursathi_trained_artifacts.zip as backup...
```